MA563 HW5 Solutions

Maggie Zhang

2026-03-31

Problem 1: Euler Methods and Stiffness in ODEs

Consider the initial value problem:

$$y' = -15y, \quad y(0) = 1$$

The exact solution is $y(t) = e^{-15t}$.

Part (a): Regions of Absolute Stability

Forward Euler Method

The forward Euler method applied to $y' = \lambda y$ gives:

$$u_{n+1} = u_n + h\lambda u_n = (1 + h\lambda)u_n$$

Thus $u_n = (1 + h\lambda)^n u_0$, and we need $|1 + h\lambda| < 1$ for $u_n \to 0$ as $n \to \infty$.

The **region of absolute stability** for forward Euler is:

$$\mathcal{R}_{FE} = \{z \in \mathbb{C} : |1 + z| < 1\}$$

This is the open disk of radius 1 centered at $z = -1$ in the complex plane.

For our problem, $\lambda = -15$, so $z = h\lambda = -15h$. We need:

$$|1 - 15h| < 1 \quad \Longrightarrow \quad -1 < 1 - 15h < 1 \quad \Longrightarrow \quad 0 < h < \frac{2}{15}$$

Backward Euler Method

The backward Euler method applied to $y' = \lambda y$ gives:

$$u_{n+1} = u_n + h\lambda u_{n+1} \quad \Longrightarrow \quad u_{n+1} = \frac{1}{1 - h\lambda} u_n$$

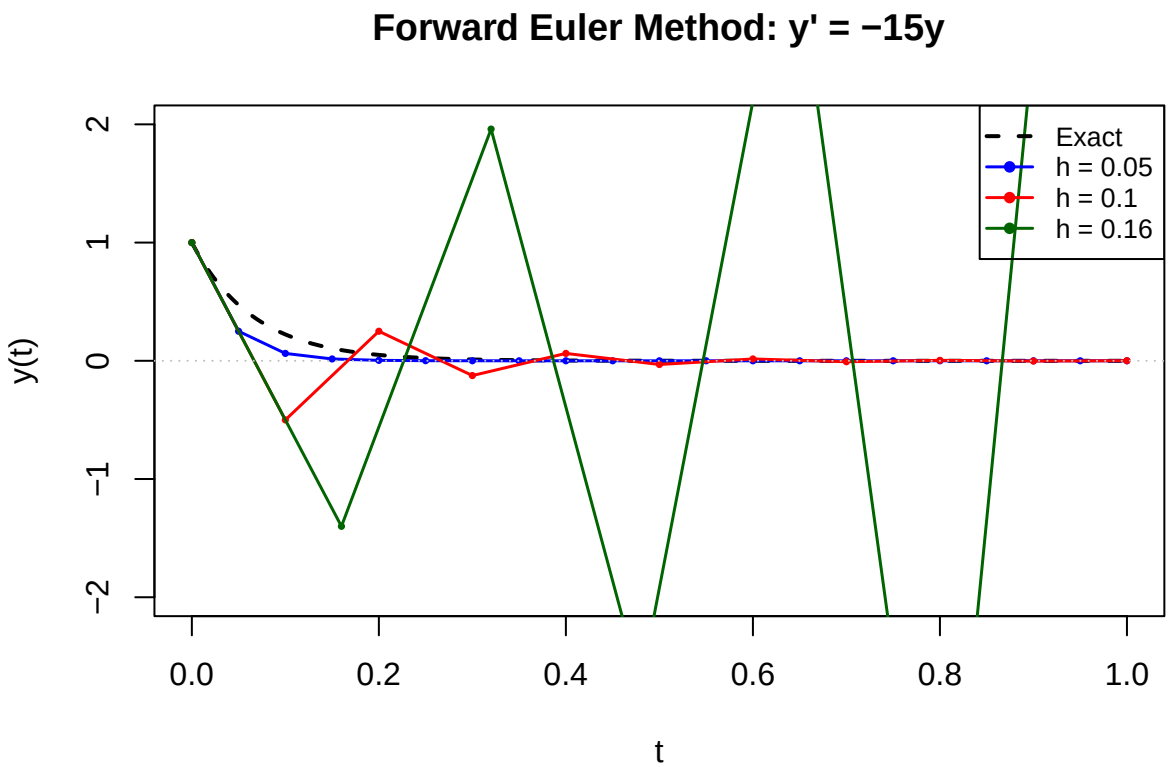
Thus $u_n = \left(\frac{1}{1-h\lambda}\right)^n u_0$, and we need $|1 - h\lambda| > 1$ for $u_n \to 0$.

The **region of absolute stability** for backward Euler is:

$$\mathcal{R}_{BE} = \{z \in \mathbb{C} : |1 - z| > 1\}$$

Part (b): Forward Euler Method

```r
forward_euler <- function(f, y0, t0, tf, h) {
  t <- seq(t0, tf, by = h)
  y <- numeric(length(t))
  y[1] <- y0
  for (i in 1:(length(t) - 1)) {
    y[i + 1] <- y[i] + h * f(t[i], y[i])
  }
  return(data.frame(t = t, y = y))
}

f <- function(t, y) -15 * y

h_values <- c(0.05, 0.1, 0.16)
colors <- c("blue", "red", "darkgreen")
t_end <- 1.0

plot(NULL, xlim = c(0, t_end), ylim = c(-2, 2),
     xlab = "t", ylab = "y(t)",
     main = "Forward Euler Method: y' = -15y")

# Exact solution
t_exact <- seq(0, t_end, length.out = 500)
lines(t_exact, exp(-15 * t_exact), col = "black", lwd = 2, lty = 2)

for (i in seq_along(h_values)) {
  sol <- forward_euler(f, 1, 0, t_end, h_values[i])
  lines(sol$t, sol$y, col = colors[i], lwd = 1.5, type = "o", pch = 16, cex = 0.5)
}

legend("topright",
       legend = c("Exact", paste0("h = ", h_values)),
       col = c("black", colors), lwd = c(2, rep(1.5, 3)),
       lty = c(2, rep(1, 3)), pch = c(NA, rep(16, 3)), cex = 0.8)

abline(h = 0, col = "gray", lty = 3)
```



Analysis:

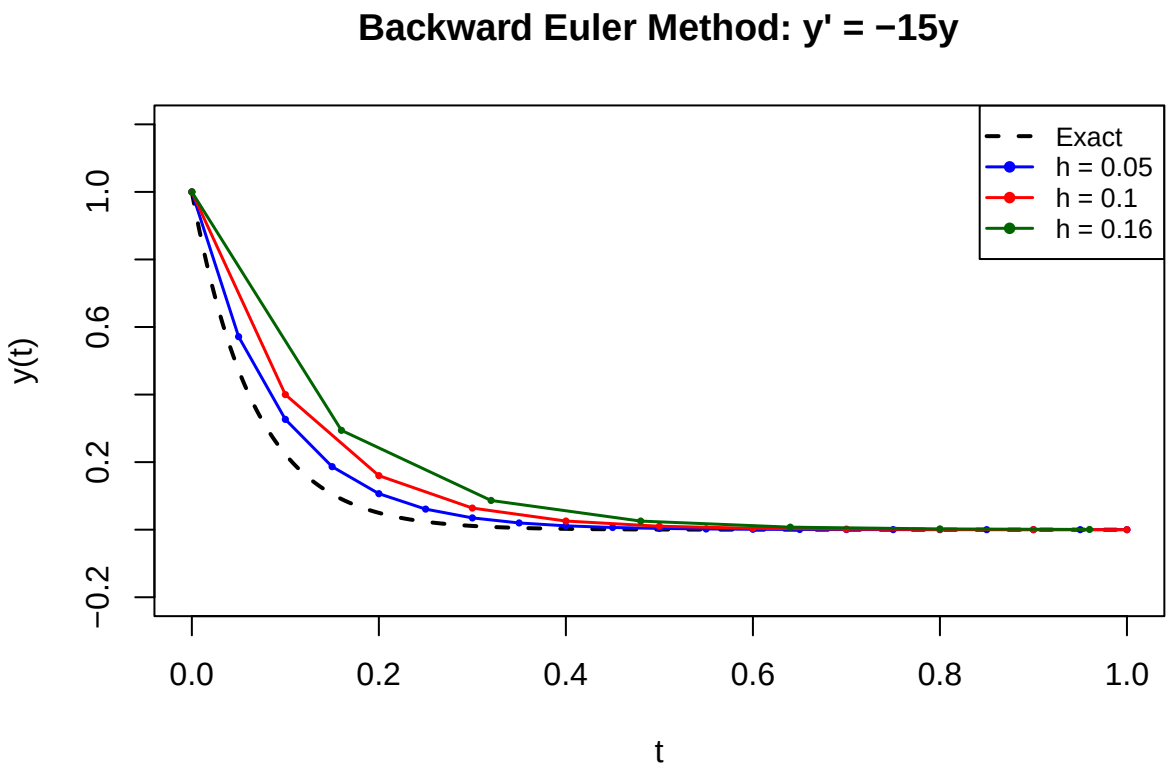
- $h = 0.05$: $z = -15 \times 0.05 = -0.75$, and $|1 + z| = |1 - 0.75| = 0.25 < 1$. The method is absolutely stable, and the numerical solution converges to the exact solution.
- $h = 0.1$: $z = -15 \times 0.1 = -1.5$, and $|1 + z| = |1 - 1.5| = 0.5 < 1$. Still absolutely stable (within the stability region), but the numerical solution oscillates in sign because $1 + z = -0.5 < 0$. The solution converges.
- $h = 0.16$: $z = -15 \times 0.16 = -2.4$, and $|1 + z| = |1 - 2.4| = 1.4 > 1$. The method is **outside the stability region**, so the numerical solution **diverges** (explodes) as $t \to \infty$.

Explanation using key concepts:

- **Consistency**: Forward Euler has LTE $= O(h)$, so it is first-order consistent for all step sizes. This property is about the local approximation quality and holds regardless of h.
- **Zero-stability**: Forward Euler is zero-stable (as proved in lecture). This ensures convergence as $h \to 0$.
- **Absolute stability**: The critical stability threshold is $h < 2/15 \approx 0.133$. For $h = 0.05$ and $h = 0.1$, $z = h\lambda$ lies inside the stability region, so the solution remains bounded. For $h = 0.16$, z lies outside, and the amplification factor $|1 + z| > 1$ causes exponential growth.

Part (c): Backward Euler Method

```r
backward_euler <- function(f_lambda, y0, t0, tf, h) {
  # For y' = lambda*y, backward Euler: u_{n+1} = u_n / (1 - h*lambda)
  lambda <- -15
  t <- seq(t0, tf, by = h)
  y <- numeric(length(t))
  y[1] <- y0
  for (i in 1:(length(t) - 1)) {
    y[i + 1] <- y[i] / (1 - h * lambda)
  }
  return(data.frame(t = t, y = y))
}

plot(NULL, xlim = c(0, t_end), ylim = c(-0.2, 1.2),
     xlab = "t", ylab = "y(t)",
     main = "Backward Euler Method: y' = -15y")

lines(t_exact, exp(-15 * t_exact), col = "black", lwd = 2, lty = 2)

for (i in seq_along(h_values)) {
  sol <- backward_euler(NULL, 1, 0, t_end, h_values[i])
  lines(sol$t, sol$y, col = colors[i], lwd = 1.5, type = "o", pch = 16, cex = 0.5)
}

legend("topright",
       legend = c("Exact", paste0("h = ", h_values)),
       col = c("black", colors), lwd = c(2, rep(1.5, 3)),
       lty = c(2, rep(1, 3)), pch = c(NA, rep(16, 3)), cex = 0.8)
```



Analysis:

For backward Euler with $\lambda = -15$, the amplification factor is:

$$g = \frac{1}{1 - h\lambda} = \frac{1}{1 + 15h}$$

- $h = 0.05$: $g = \frac{1}{1.75} \approx 0.571$. Stable and accurate.
- $h = 0.1$: $g = \frac{1}{2.5} = 0.4$. Stable, slightly less accurate but bounded.
- $h = 0.16$: $g = \frac{1}{3.4} \approx 0.294$. Stable, less accurate but still bounded.

All three solutions **converge to the exact solution** as $h \to 0$, and all remain **bounded** as $t \to \infty$.

Explanation using key concepts:

- **Consistency**: Backward Euler has LTE $= O(h)$, so it is first-order consistent.
- **Zero-stability**: Backward Euler is zero-stable.
- **Absolute stability**: Backward Euler is **A-stable**: its stability region $\mathcal{R}_{BE} = \{z : |1-z| > 1\}$ contains the entire left half-plane $\mathbb{C}^-$. Since $\lambda = -15 < 0$, we have $z = -15h < 0$ for all $h > 0$, which is always in $\mathcal{R}_{BE}$. There is **no stability constraint** on h, so the solution is bounded for any step size.

The only constraint on h for backward Euler is **accuracy** (smaller h gives better approximation), not stability. This makes backward Euler ideal for **stiff problems** like this one where $|\lambda| = 15$ is moderately large.

Problem 2: Family of One-Step Methods

Consider:

$$u_{n+1} = u_n + h\left[\left(1 - \frac{\alpha}{2}\right) f(t_n, u_n) + \frac{\alpha}{2} f(t_{n+1}, u_{n+1})\right]$$

Part (a): Order of Consistency

To determine the order of consistency, we substitute the exact solution $y(t)$ into the scheme and compute the local truncation error.

The Local Truncation Error (LTE) is defined by:

$$f(t_n, y_n) = \left(1 - \frac{\alpha}{2}\right) f(t_n, y_n) + \frac{\alpha}{2} f(t_{n+1}, y_{n+1})$$

$$\tau_{n+1} = \frac{y_{n+1} - y_n}{h} - f(t_n, y_n)$$

Expand the LHS using Taylor series around t_n:

$$\frac{y_{n+1} - y_n}{h} = y_n' + \frac{h}{2} y_n'' + \frac{h^2}{6} y_n''' + O(h^3)$$

Expand the RHS: Since $y' = f(t, y)$, we have $f(t_n, y_n) = y_n'$ and

$$f(t_{n+1}, y_{n+1}) = y_{n+1}' = y_n' + h y_n'' + \frac{h^2}{2} y_n''' + O(h^3)$$

Therefore:

$$\begin{aligned} \text{RHS} &= \left(1 - \frac{\alpha}{2}\right) y_n' + \frac{\alpha}{2}\left(y_n' + h y_n'' + \frac{h^2}{2} y_n''' + O(h^3)\right) \\ &= y_n' + \frac{\alpha}{2} h y_n'' + \frac{\alpha}{4} h^2 y_n''' + O(h^3) \end{aligned}$$

LTE:

$$\begin{aligned} \tau_{n+1} &= \left(y_n' + \frac{h}{2} y_n'' + \frac{h^2}{6} y_n''' + O(h^3)\right) - \left(y_n' + \frac{\alpha}{2} h y_n'' + \frac{\alpha}{4} h^2 y_n''' + O(h^3)\right) \\ &= \left(\frac{1}{2} - \frac{\alpha}{2}\right) h y_n'' + \left(\frac{1}{6} - \frac{\alpha}{4}\right) h^2 y_n''' + O(h^3) \end{aligned}$$

Analysis by cases:

- **If** $\alpha \neq 1$: The leading term is $\left(\frac{1-\alpha}{2}\right) h y_n'' \neq 0$, so $\tau_{n+1} = O(h)$. The method is **first-order consistent**.
- **If** $\alpha = 1$: The $O(h)$ term vanishes ($\frac{1-1}{2} = 0$), and the leading term becomes $\left(\frac{1}{6} - \frac{1}{4}\right) h^2 y_n''' = -\frac{1}{12} h^2 y_n'''$, so $\tau_{n+1} = O(h^2)$. The method is **second-order consistent**.

Summary: The method is order 1 for $\alpha \neq 1$ and order 2 for $\alpha = 1$ (which is the **trapezoidal method**).

Part (b): Region of Absolute Stability for $\alpha = 1$

When $\alpha = 1$, the method becomes:

$$u_{n+1} = u_n + h\left[\frac{1}{2}f(t_n, u_n) + \frac{1}{2}f(t_{n+1}, u_{n+1})\right]$$

This is the **trapezoidal method**. Apply to the test equation $y' = \lambda y$ ($f(t, y) = \lambda y$):

$$\begin{aligned} u_{n+1} &= u_n + h\left[\frac{1}{2}\lambda u_n + \frac{1}{2}\lambda u_{n+1}\right] \\ u_{n+1} - \frac{h\lambda}{2}u_{n+1} &= u_n + \frac{h\lambda}{2}u_n \\ u_{n+1}\left(1 - \frac{z}{2}\right) &= u_n\left(1 + \frac{z}{2}\right) \end{aligned}$$

where $z = h\lambda$. The stability function is:

$$R(z) = \frac{1 + z/2}{1 - z/2}$$

The **region of absolute stability** is:

$$\mathcal{R} = \left\{z \in \mathbb{C} : \left|\frac{1 + z/2}{1 - z/2}\right| < 1\right\}$$

We need $|1 + z/2| < |1 - z/2|$. Let $z = x + iy$:

$$\begin{aligned} |1 + z/2|^2 &< |1 - z/2|^2 \\ (1 + x/2)^2 + (y/2)^2 &< (1 - x/2)^2 + (y/2)^2 \\ 1 + x + x^2/4 &< 1 - x + x^2/4 \\ 2x < 0 \quad &\Longrightarrow \quad x < 0 \end{aligned}$$

Therefore: $\mathcal{R} = \{z \in \mathbb{C} : \mathrm{Re}(z) < 0\} = \mathbb{C}^-$

(The trapezoidal method is **A-stable**: its stability region is the entire open left half-plane.)

Problem 3: Trade-offs Between Order and Stability in RK Methods

Part (a): Two-Stage Second-Order RK Method

Consider:

$$\begin{aligned} f_1 = f(u_n), \quad f_2 &= f(u_n + ahf_1) \\ u_{n+1} &= u_n + h\left[\left(1 - \frac{1}{2a}\right)f_1 + \frac{1}{2a}f_2\right] \end{aligned}$$

(1) Stability Function $R(z)$

Apply to the test equation $y' = \lambda y$, let $z = h\lambda$:

$$f_1 = \lambda u_n$$
$$f_2 = \lambda(u_n + ah\lambda u_n) = \lambda u_n(1 + az)$$

$$u_{n+1} = u_n + h\left[\left(1 - \frac{1}{2a}\right)\lambda u_n + \frac{1}{2a}\lambda u_n(1 + az)\right]$$

$$= u_n + zu_n\left[\left(1 - \frac{1}{2a}\right) + \frac{1}{2a}(1 + az)\right]$$

$$= u_n + zu_n\left[1 - \frac{1}{2a} + \frac{1}{2a} + \frac{z}{2}\right]$$

$$= u_n\left[1 + z + \frac{z^2}{2}\right]$$

Therefore: $u_{n+1} = q(z)u_n$ where:

$$\boxed{q(z) = 1 + z + \frac{1}{2}z^2}$$

(2) Independence of a

As shown above, in the derivation all terms involving a cancel:

$$\left(1 - \frac{1}{2a}\right) + \frac{1}{2a} = 1$$

and the z^2 coefficient comes from $\frac{1}{2a} \cdot az = \frac{z}{2}$, which is also independent of a.

This is because any second-order two-stage RK method must match the Taylor expansion $e^z = 1 + z + \frac{z^2}{2} + O(z^3)$ through the z^2 term, and the second-order conditions uniquely determine $q(z)$ regardless of how the free parameter a is chosen.

(3) Real-Axis Stability Interval

We need to find the interval $(-b, 0)$ on the real axis where $|q(z)| \leq 1$ for real $z < 0$.

For real z: $q(z) = 1 + z + \frac{z^2}{2}$.

We need $-1 \leq q(z) \leq 1$.

Upper bound $q(z) \leq 1$: $z + \frac{z^2}{2} \leq 0 \implies z(1 + z/2) \leq 0$. For $z < 0$: need $1 + z/2 \geq 0$, i.e., $z \geq -2$. So $q(z) \leq 1$ for $z \in [-2, 0]$.

Lower bound $q(z) \geq -1$: $2 + z + \frac{z^2}{2} \geq 0 \implies z^2 + 2z + 4 \geq 0$. The discriminant is $4 - 16 = -12 < 0$, so this quadratic is **always positive**. Thus $q(z) \geq -1$ for all real z.

Therefore: the real-axis stability interval is $\boxed{(-2, 0)}$, i.e., $b = 2$.

Part (b): First-Order RK with Extended Stability

Now consider $q(z) = 1 + z + cz^2$ for some $c > 0$ (first-order accurate since the $z^2/2$ coefficient is replaced by $c \neq 1/2$).

We want $|q(z)| \leq 1$ on the real negative axis.

Upper bound $q(z) \leq 1$: $z + cz^2 \leq 0 \implies z(1 + cz) \leq 0$. For $z < 0$: need $1 + cz \geq 0$, i.e., $z \geq -1/c$. So $q(z) \leq 1$ for $z \in [-1/c, 0]$.

Lower bound $q(z) \geq -1$: $cz^2 + z + 2 \geq 0$. The discriminant is $1 - 8c$.

- If $c \geq 1/8$: discriminant ≤ 0, so $cz^2 + z + 2 > 0$ for all real z. Thus $q(z) \geq -1$ always.
- If $c < 1/8$: the quadratic has real roots, and $q(z) = -1$ at $z = \frac{-1 \pm \sqrt{1-8c}}{2c}$.

When $c \geq 1/8$, the stability interval is determined only by the upper bound:

$$(-b, 0) = \left(-\frac{1}{c}, 0\right), \quad b = \frac{1}{c}$$

To extend the stability interval to $(-4, 0)$, we need $b \geq 4$, i.e., $\frac{1}{c} \geq 4$, i.e., $c \leq 1/4$.

We also need $c \geq 1/8$ to ensure the lower bound holds everywhere.

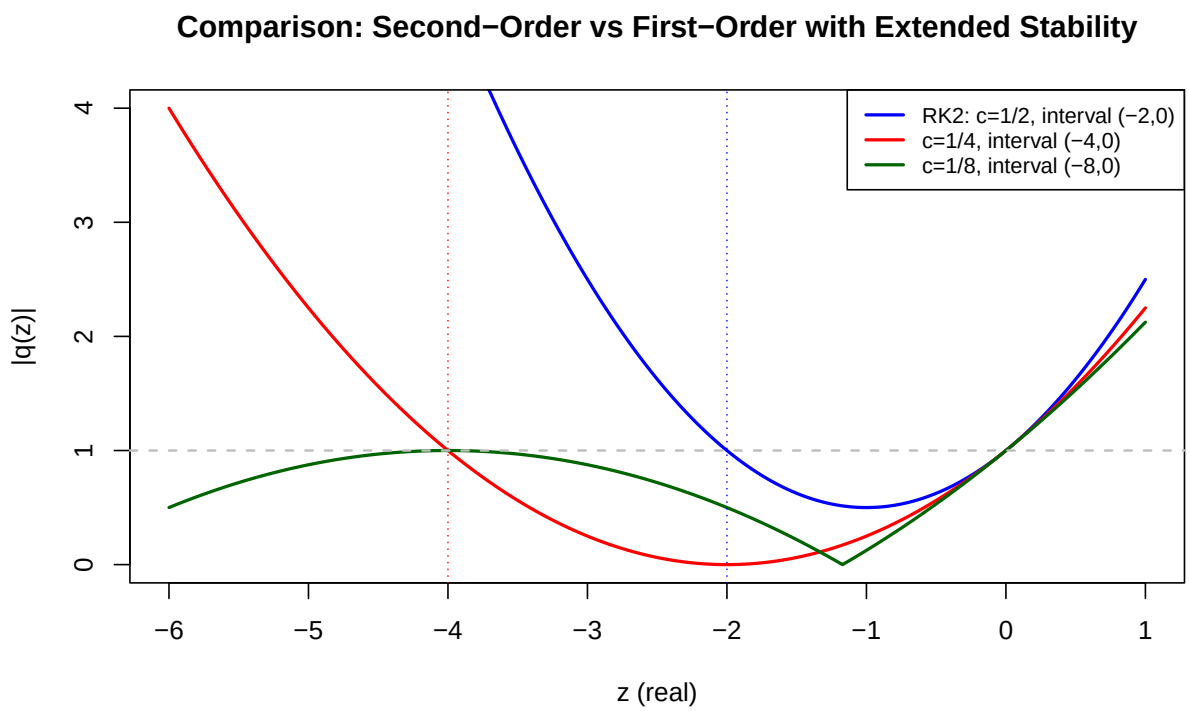
Optimal choice: $c = 1/4$ gives $b = 4$ and $c \geq 1/8$. So:

$$q(z) = 1 + z + \frac{1}{4}z^2$$

has stability interval $(-4, 0)$, which is **twice as long** as the second-order method's interval $(-2, 0)$.

Verification: at $z = -4$: $q(-4) = 1 - 4 + \frac{1}{4}(16) = 1 - 4 + 4 = 1$, so $|q(-4)| = 1$. For $z \in (-4, 0)$: $|q(z)| < 1$.

```r
z <- seq(-6, 1, length.out = 1000)

q_rk2 <- 1 + z + z^2 / 2
q_c14 <- 1 + z + 0.25 * z^2
q_c18 <- 1 + z + 0.125 * z^2

plot(z, abs(q_rk2), type = "l", col = "blue", lwd = 2,
     xlab = "z (real)", ylab = "|q(z)|", ylim = c(0, 4),
     main = "Comparison: Second-Order vs First-Order with Extended Stability")
lines(z, abs(q_c14), col = "red", lwd = 2)
lines(z, abs(q_c18), col = "darkgreen", lwd = 2)
abline(h = 1, col = "gray", lty = 2, lwd = 1.5)
abline(v = c(-2, -4, -8), col = c("blue", "red", "darkgreen"), lty = 3)
legend("topright",
       legend = c("RK2: c=1/2, interval (-2,0)",
                  "c=1/4, interval (-4,0)",
                  "c=1/8, interval (-8,0)"),
       col = c("blue", "red", "darkgreen"), lwd = 2, cex = 0.8)
```



Part (c): Interpretation of the Trade-off

Gained: By relaxing the second-order accuracy requirement, we gain freedom to choose the parameter c to enlarge the stability region. With $c = 1/4$, the real-axis stability interval doubles from $(-2, 0)$ to $(-4, 0)$. This means we can use a step size twice as large while maintaining absolute stability.

Lost: The method drops from second-order to first-order accuracy. This means the local truncation error is $O(h)$ instead of $O(h^2)$, and for a given step size, the numerical solution will be less accurate.

When is this trade-off advantageous?

This trade-off is advantageous for **stiff problems** where:

1. **Stability dominates accuracy**: The step size h is primarily limited by stability constraints rather than accuracy requirements. In stiff problems, the eigenvalues of the Jacobian have large negative real parts, requiring very small h for an explicit method to remain stable. By extending the stability interval, we can use a larger h.

2. **Moderate accuracy suffices**: When the solution varies slowly (after initial transients decay), high-order accuracy is not critical. A first-order method with a larger stable step size may be more efficient than a second-order method forced to use a tiny step size.

3. **Computational cost per step is important**: If function evaluations are expensive, being able to take larger steps (even with lower order) can reduce overall computational cost.

Overall, for stability-dominated problems (stiff ODEs), it can be more efficient to use a lower-order method with a larger stability region than a higher-order method with a smaller stability region. This is the fundamental principle behind stabilized explicit Runge-Kutta methods.